

Politechnika Wrocławska

Układy Cyfrowe i Systemy Wbudowane 1

Laboratorium

Sprawozdanie z ćwiczenia 1

Czterobitowy układ kombinacyjny

wariant c): $Y = (4 - X) \bmod 16$

Wykonali:	Adam Korwin, 284455 Dawid Kasiarz, 284541
Prowadzący:	dr inż. Jarosław Sugier
Platforma:	AMD Vivado, płyta AUP-ZU3 (`xczu3eg-sfvc784-2-e`)
Data zajęć:	22.10.2026

1 Treść zadania

Celem ćwiczenia było zaprojektowanie czterobitowego układu kombinacyjnego, który dla wejścia X wyznacza wartość $Y = (4 - X) \bmod 16$. Oba wektory, `X(3 downto 0)` oraz `Y(3 downto 0)`, są zapisane w naturalnym kodzie binarnym (NKB). Ćwiczenie składało się z czterech zadań:

Z1 minimalizacja funkcji $Y(0)$ do $Y(3)$ w tablicach Karnaugha, opis układu w VHDL i symulacja behawioralna dla wszystkich wartości X od 0 do 15, zmienianych co 100 ns;

Z2 uruchomienie układu na płycie, gdzie X pochodzi z przełączników `PL_USER_SW(3:0)`, a Y steruje diodami `PL_USER_LED(3:0)`, oraz symulacja czasowa przełączania bitów Y przy zmianie $Y : 8 \to 7$;

Z3 diagram blokowy, w którym układ z Z1 jest osobnym modułem, a wartości X i Y są pokazywane na wyświetlaczu OLED (`OLED_1x32`);

Z4 realizacja tego samego działania przy pomocy gotowego bloku IP *Adder/Subtracter* i porównanie obu wyników na wyświetlaczu.

2 Z1: projekt układu i symulacja behawioralna

2.1 Tablica prawdy

Dla $X \leq 4$ wynik jest po prostu różnicą $4 - X$. Dla $X \geq 5$ różnica jest ujemna, więc do wyniku dodajemy 16. Pełną tablicę zawiera tabela 1.

X	$x_3x_2x_1x_0$	Y	$y_3y_2y_1y_0$	X	$x_3x_2x_1x_0$	Y	$y_3y_2y_1y_0$
0	0000	4	0100	8	1000	12	1100
1	0001	3	0011	9	1001	11	1011
2	0010	2	0010	10	1010	10	1010
3	0011	1	0001	11	1011	9	1001
4	0100	0	0000	**12**	**1100**	**8**	**1000**
5	0101	15	1111	**13**	**1101**	**7**	**0111**
6	0110	14	1110	14	1110	6	0110
7	0111	13	1101	15	1111	5	0101

Tabela 1: Tablica prawdy układu. Pogrubione wiersze odpowiadają zmianie $Y : 8 \to 7$ badanej w zadaniu Z2.

2.2 Minimalizacja w tablicach Karnaugha

W mapach wiersze opisują x_3x_2, a kolumny x_1x_0, obie osie w kodzie Graya. Pola z jedynkami są wyróżnione kolorem.

$Y(3)$

$x_3x_2\backslash x_1x_0$	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	1	0	0	0
10	1	1	1	1

$Y(2)$

$x_3x_2\backslash x_1x_0$	00	01	11	10
00	1	0	0	0
01	0	1	1	1
11	0	1	1	1
10	1	0	0	0

$Y(1)$

$x_3x_2\backslash x_1x_0$	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

$Y(0)$

$x_3x_2\backslash x_1x_0$	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

Rysunek 1: Tablice Karnaugha dla bitów wyniku.

Odczytane grupy:

$Y(0)$**.** Jedynki zajmują dwie środkowe kolumny, czyli wszystkie pola z $x_0 = 1$. Bit najmłodszy jest więc kopią wejścia. Wynika to z tego, że odejmowanie od liczby parzystej nie zmienia parzystości.

$Y(1)$**.** Jedynki leżą w kolumnach 01 i 10, które nie są ze sobą sąsiednie. Powstają dwie grupy po osiem pól, $x_1\overline{x}_0$ oraz $\overline{x}_1x_0$, czyli funkcja XOR.

$Y(2)$**.** Grupa x_2x_1 obejmuje wiersze 01 i 11 w kolumnach 11 i 10. Grupa x_2x_0 obejmuje te same wiersze w kolumnach 01 i 11. Ostatnia grupa, $\overline{x}_2\overline{x}_1\overline{x}_0$, łączy pola $X = 0$ oraz $X = 8$, bo skrajne wiersze mapy są sąsiednie.

$Y(3)$**.** Cały wiersz 10 daje $x_3\overline{x}_2$. Pola $X = 8$ i $X = 12$ w kolumnie 00 dają $x_3\overline{x}_1\overline{x}_0$. W wierszu 01 pozostają pola $X = 5, 6, 7$, które pokrywamy grupami $\overline{x}_3x_2x_1$ oraz $\overline{x}_3x_2x_0$.

Otrzymane równania:

$$
\begin{aligned}
Y(0) &= x_0 \\
Y(1) &= x_1\overline{x}_0 + \overline{x}_1x_0 = x_1 \oplus x_0 \\
Y(2) &= x_2x_1 + x_2x_0 + \overline{x}_2\overline{x}_1\overline{x}_0 = \overline{x_2 \oplus (x_1 + x_0)} \\
Y(3) &= x_3\overline{x}_2 + x_3\overline{x}_1\overline{x}_0 + \overline{x}_3x_2x_1 + \overline{x}_3x_2x_0
\end{aligned}
$$

Poprawność równań sprawdziliśmy dodatkowo krótkim skryptem w Pythonie, który porównuje wynik równań z wartością $(4 - X) \bmod 16$ dla wszystkich szesnastu wartości X.

2.3 Opis w VHDL

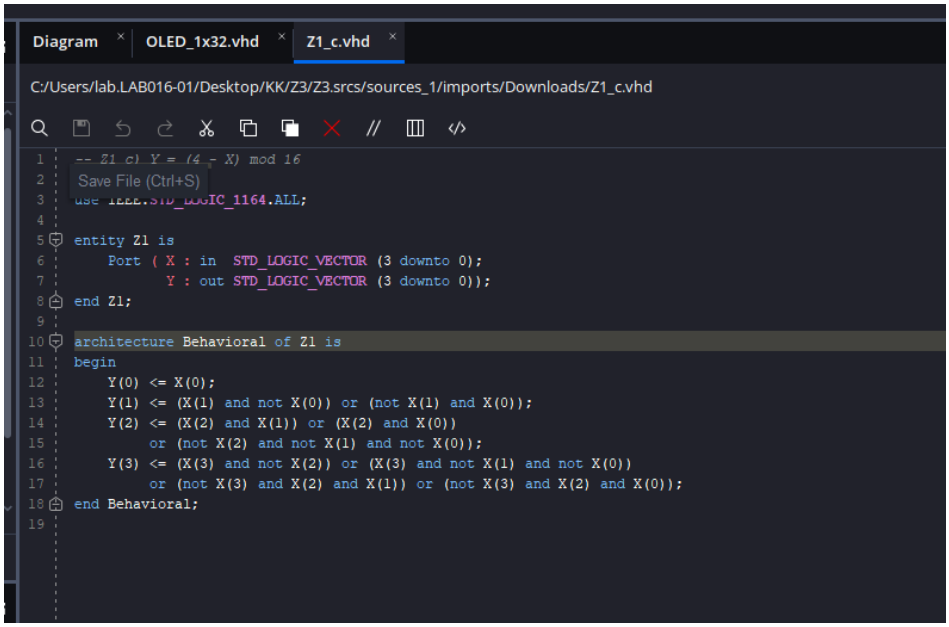
Równania zapisaliśmy bezpośrednio jako przypisania współbieżne w encji `Z1`. Przy mieszaniu operatorów `and` i `or` każdy iloczyn jest ujęty w nawiasy, ponieważ VHDL nie nadaje tym operatorom różnych priorytetów.

Listing 1: Plik `Z1_c.vhd`

```
-- Z1 c) Y = (4 - X) mod 16
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Z1 is
    Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
           Y : out STD_LOGIC_VECTOR (3 downto 0));
end Z1;
```

```
9
10 architecture Behavioral of Z1 is
11 begin
12     Y(0) <= X(0);
13     Y(1) <= (X(1) and not X(0)) or (not X(1) and X(0));
14     Y(2) <= (X(2) and X(1)) or (X(2) and X(0))
15          or (not X(2) and not X(1) and not X(0));
16     Y(3) <= (X(3) and not X(2)) or (X(3) and not X(1) and not X(0))
17          or (not X(3) and X(2) and X(1)) or (not X(3) and X(2) and X(0));
18 end Behavioral;
```



Rysunek 2: Plik `Z1_c.vhd` otwarty w edytorze Vivado na zajęciach.

2.4 Testbench i wynik symulacji

Testbench podaje na wejście kolejne wartości X co 100 ns, dokładnie w sposób podany w poleceniu (`X <= "0000", "0001"after 100 ns, ...`). Dodaliśmy do niego proces `Check`, który w połowie każdego kroku porównuje Y z wartością oczekiwaną i w razie niezgodności wypisuje komunikat `Blad dla X = ....`

Listing 2: Plik `Z1_tb.vhd`

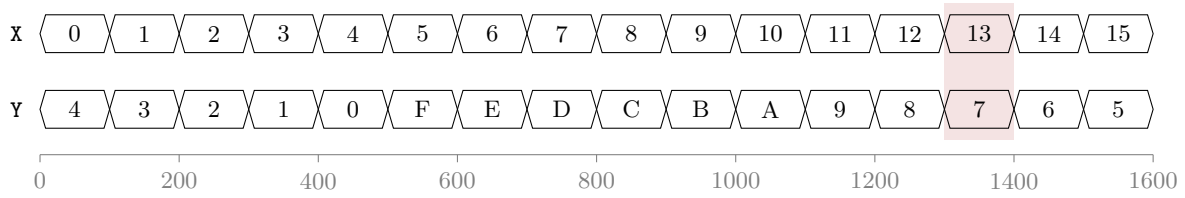
```
1  -- Testbench Z1 c): X = 0..15, co 100 ns. Zmiana Y: 8 -> 7 nastepuje w 1300 ns (X: 12
       -> 13).
2  library IEEE;
3  use IEEE.STD_LOGIC_1164.ALL;
4  use IEEE.NUMERIC_STD.ALL;
5
6  entity Z1_tb is
7  end Z1_tb;
8
9  architecture Behavioral of Z1_tb is
10     component Z1
11         Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
12                Y : out STD_LOGIC_VECTOR (3 downto 0));
13     end component;
14
15     signal X : STD_LOGIC_VECTOR (3 downto 0);
```

```
    signal Y : STD_LOGIC_VECTOR (3 downto 0);
begin
    UUT: Z1 port map ( X => X, Y => Y );

    X <= "0000",
         "0001" after  100 ns, "0010" after  200 ns, "0011" after  300 ns,
         "0100" after  400 ns, "0101" after  500 ns, "0110" after  600 ns,
         "0111" after  700 ns, "1000" after  800 ns, "1001" after  900 ns,
         "1010" after 1000 ns, "1011" after 1100 ns, "1100" after 1200 ns,
         "1101" after 1300 ns, "1110" after 1400 ns, "1111" after 1500 ns;

    -- Samosprawdzenie (opcjonalne): w polowie kazdego kroku Y musi byc (4 - X) mod 16
    Check: process
    begin
        for i in 0 to 15 loop
            wait for 50 ns;
            assert to_integer(unsigned(Y)) = (4 - i) mod 16
                report "Blad dla X = " & integer'image(i) severity error;
            wait for 50 ns;
        end loop;
        report "Koniec testu";
        wait;
    end process;
end Behavioral;
```

Symulacja zakończyła się komunikatem `Koniec testu` w chwili 1600 ns i nie pojawił się żaden komunikat o błędzie. Przebiegi z symulacji pokazuje rysunek 3.



Rysunek 3: Symulacja behawioralna: X dziesiętnie, Y szesnastkowo, oś czasu w ns. Zaznaczony przedział od 1300 ns to $X = 13$, $Y = 7$.

3 Z2: uruchomienie na płycie i symulacja czasowa

3.1 Przypisanie wyprowadzeń

Do projektu z Z1 dodaliśmy plik ograniczeń, w którym porty `X` i `Y` są przypisane do przełączników SW0 do SW3 i diod LD0 do LD3. Numery wyprowadzeń przepisaliśmy z pliku `ZU3.xdc` udostępnionego przez prowadzącego. Wszystkie te linie pracują w standardzie `LVCMOS12`.

Listing 3: Plik `Z2.xdc`

```
## Z2: X(3:0) <- PL_USER_SW(3:0), Y(3:0) -> PL_USER_LED(3:0)  (linie z ZU3.xdc)

set_property PACKAGE_PIN AB1 [get_ports {X[0]}]
set_property PACKAGE_PIN AF1 [get_ports {X[1]}]
set_property PACKAGE_PIN AE3 [get_ports {X[2]}]
set_property PACKAGE_PIN AC2 [get_ports {X[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports X*]

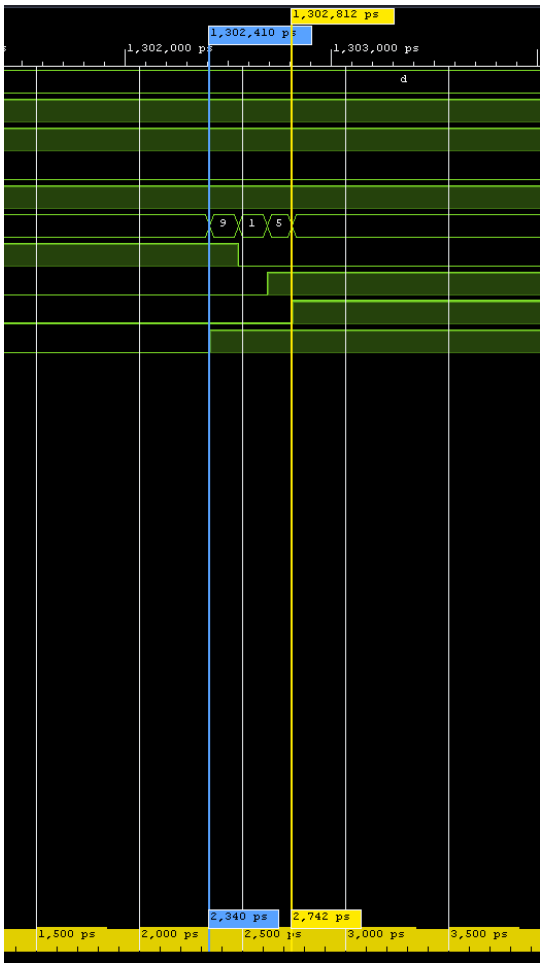
set_property PACKAGE_PIN AF5 [get_ports {Y[0]}]
```

```
10 set_property PACKAGE_PIN AE7 [get_ports {Y[1]}]
11 set_property PACKAGE_PIN AH2 [get_ports {Y[2]}]
12 set_property PACKAGE_PIN AE5 [get_ports {Y[3]}]
13 set_property IOSTANDARD LVCMOS12 [get_ports Y*]
```

Po syntezie, implementacji i wygenerowaniu pliku `.bit` zaprogramowaliśmy układ przez Hardware Manager. Działanie sprawdziliśmy, ustawiając na przełącznikach różne wartości X i porównując stan diod z tabelą 1.

3.2 Symulacja czasowa dla zmiany $Y: 8 \to 7$

Wartość $Y = 8$ odpowiada $X = 12$, a $Y = 7$ odpowiada $X = 13$. W testbenchu ta zmiana następuje w chwili 1300 ns. Na wejściu zmienia się tylko jeden bit ($x_0: 0 \to 1$), natomiast na wyjściu zmieniają się wszystkie cztery ($1000 \to 0111$). Domyślny czas symulacji w Vivado wynosi 1000 ns, więc w konsoli Tcl wydaliśmy dodatkowo polecenie `run 500 ns`. Symulację uruchomiliśmy w trybie *Post-Implementation Timing Simulation*, w którym uwzględnione są opóźnienia bramek i połączeń po rozmieszczeniu układu w FPGA.



Rysunek 4: Symulacja czasowa w okolicy 1302 ns. Górny wiersz to szyna X (wartość `d`), niżej szyna Y z wartościami przejściowymi, a pod nią bity $Y(3)$, $Y(2)$, $Y(1)$, $Y(0)$. Kursory: 1 302 410 ps i 1 302 812 ps.

Na rysunku 4 widać, że bity wyjścia nie zmieniają się jednocześnie. Kolejność zmian odczytana z przebiegu jest zebrana w tabeli 2.

czas [ps]	zmieniający się bit	$y_3y_2y_1y_0$	Y
do 1 302 410	stan początkowy	1000	8
1 302 410	$Y(0)\colon 0 \to 1$	1001	9
ok. 1 302 555	$Y(3)\colon 1 \to 0$	0001	1
ok. 1 302 695	$Y(2)\colon 0 \to 1$	0101	5
1 302 812	$Y(1)\colon 0 \to 1$	0111	7

Tabela 2: Kolejność przełączania bitów wyjścia. Czasy środkowych zmian są odczytane z podziałki z dokładnością kilkunastu pikosekund.

Pierwsza zmiana na wyjściu pojawia się około 2,4 ns po zmianie wejścia. Ten czas obejmuje przejście przez bufor wejściowy, połączenia wewnątrz FPGA i bufor wyjściowy. Najszybciej reaguje $Y(0)$, bo jest to zwykłe połączenie wejścia x_0 z wyjściem i sygnał nie przechodzi przez żadną tablicę LUT. Pozostałe bity są liczone w tablicach LUT, a ich ścieżki mają różną długość, dlatego przełączają się później i w różnych chwilach.

W efekcie przez około 400 ps na wyjściu pojawiają się kolejno wartości 9, 1 i 5, których nie ma w tablicy prawdy dla żadnego z dwóch stanów wejścia. Jest to hazard wynikający z różnych opóźnień ścieżek. Każdy pojedynczy bit zmienia się w tym przypadku tylko raz, więc na żadnej linii nie widać krótkiej szpilki, błędny jest dopiero cały wektor. Na diodach takie zjawisko jest niewidoczne, ale gdyby Y był wejściem dla układu synchronicznego, trzeba by próbkować go dopiero po ustaleniu się wszystkich bitów, czyli z okresem zegara dłuższym niż najdłuższe opóźnienie ścieżki.

4 Z3: diagram blokowy z wyświetlaczem OLED

W nowym projekcie utworzyliśmy diagram blokowy (*IP Integrator*). Plik `Z1_c.vhd` dodaliśmy do diagramu jako moduł RTL (`Z1_0`), a obok umieściliśmy moduł `OLED_1x32` od prowadzącego. Wyświetlacz pokazuje osiem cyfr szesnastkowych przekazanych przez 32-bitowe wejście `Line`. Wejście `Blank` określa, które cyfry są ukryte (jedynka ukrywa cyfrę).

Wejście `Line` złożyliśmy blokiem *Inline Concat*. Na najmłodsze bity (prawa cyfra wyświetlacza) trafił wynik Y, na najstarsze (lewa cyfra) wejście X z przełączników, a pozostałe bity wypełniła stała zero. Wartość `Blank` = 126 (`01111110`) ukrywa sześć środkowych cyfr, więc na wyświetlaczu widać układ `X Y`.

Moduł OLED potrzebuje zegara 100 MHz. Dostarcza go blok *Clocking Wizard* podłączony do różnicowego zegara płyty (*Board*, *Differential Clock*, *Auto Connect*). Z tego powodu w pliku XDC nie ma linii opisujących zegar. Wyjście `I2C_NACK` wyprowadziliśmy na diodę LD7, która świeci przy błędzie komunikacji z wyświetlaczem. Porty utworzone poleceniem *Make External* otrzymują przyrostek `_0`, więc zmieniliśmy ich nazwy tak, by zgadzały się z plikiem XDC.

Listing 4: Plik `Z3_Z4.xdc` (wspólny dla Z3 i Z4)

```
## Z3/Z4 (diagram blokowy): nazwy portow jak w diagramie.
## Zegara CLK_100MHz_* tu NIE opisujemy -- robi to Clocking Wizard.

## Przelaczniki SW(3:0) -> port PL_USER_SW[3:0]
set_property PACKAGE_PIN AB1 [get_ports {PL_USER_SW[0]}]
set_property PACKAGE_PIN AF1 [get_ports {PL_USER_SW[1]}]
set_property PACKAGE_PIN AE3 [get_ports {PL_USER_SW[2]}]
set_property PACKAGE_PIN AC2 [get_ports {PL_USER_SW[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports PL_USER_SW*]

## LED7 <- NACK (swieci = blad komunikacji z OLED)
set_property PACKAGE_PIN AF2 [get_ports {NACK}]
set_property IOSTANDARD LVCMOS12 [get_ports NACK]
```

```
14
15 ## OLED IIC (plyta rozszerzajaca, linie z ZU3_Exp.xdc)
16 set_property PACKAGE_PIN AE12 [get_ports {OLED_SCL}]
17 set_property PACKAGE_PIN AF12 [get_ports {OLED_SDA}]
18 set_property IOSTANDARD LVCMOS33 [get_ports OLED_*]
```

5 Z4: realizacja blokiem Adder/Subtracter

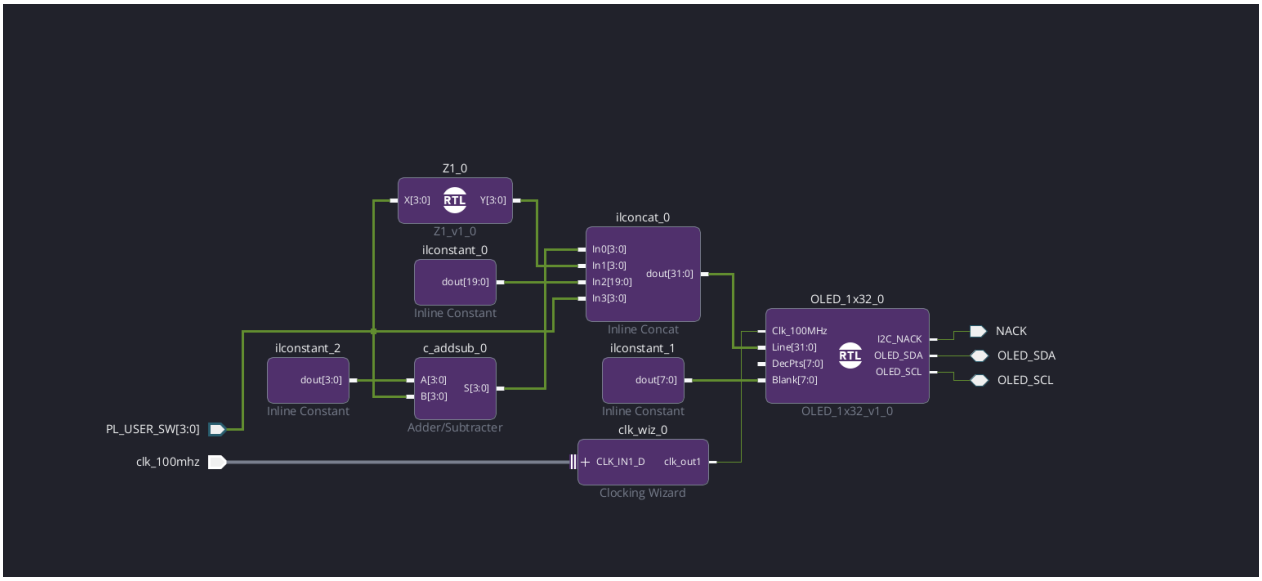
Do diagramu z Z3 dodaliśmy blok IP *Adder/Subtracter* (`c_addsub_0`) z ustawieniami podanymi w tabeli 3.

parametr	wartość
Implement using	Fabric
Add mode	Subtract
Input type, szerokość A i B	Unsigned, 4 bity
Output width	4 bity
Latency	0 (układ czysto kombinacyjny)
Clock Enable (zakładka Control)	wyłączone

Tabela 3: Konfiguracja bloku `c_addsub_0`.

Na wejście A podaliśmy stałą 4 (`ilconstant_2`), a na wejście B wektor X z przełączników. Wyjście bloku ma tylko 4 bity, więc pożyczka z najstarszej pozycji jest odrzucana i wynik odejmowania jest automatycznie liczony modulo 16. Opóźnienie ustawiliśmy na zero, żeby blok nie wymagał zegara i zachowywał się tak samo jak moduł `Z1`.

Wynik `S` dołączyliśmy do bloku Concat jako dodatkową cyfrę. Kolejność wejść: `In0` = S (4 bity), `In1` = Y (4 bity), `In2` = stała 0 (20 bitów), `In3` = X (4 bity). Wartość `Blank` zmieniliśmy na 124 (`01111100`), dzięki czemu widoczne są trzy cyfry i wyświetlacz pokazuje `X YS`. Gotowy diagram przedstawia rysunek 5.



Rysunek 5: Diagram blokowy z zadania Z4. Moduł `Z1_0` i subtraktor `c_addsub_0` liczą ten sam wynik, a `OLED_1x32_0` wyświetla X, Y oraz S.

Dla każdego ustawienia przełączników dwie prawe cyfry na wyświetlaczu były takie same, czyli moduł zaprojektowany ręcznie i blok IP dają identyczny wynik.

6 Wnioski

Minimalizacja w tablicach Karnaugha dała proste równania. Najmłodszy bit wyniku okazał się zwykłym przewodem, a drugi bit funkcją XOR, co dobrze widać na mapach. Symulacja behawioralna potwierdziła poprawność układu dla wszystkich szesnastu wartości wejścia.

Symulacja czasowa pokazała, że nawet przy zmianie jednego bitu na wejściu wyjście nie przechodzi od razu do nowej wartości. Przy zmianie Y z 8 na 7 przez około 400 ps pojawiały się wartości pośrednie 9, 1 i 5. W układzie sterującym diodami nie ma to znaczenia, ale przy dalszym przetwarzaniu wyniku przez logikę synchroniczną trzeba uwzględnić czas ustalania się wyjścia.

Gotowy blok *Adder/Subtracter* pozwolił uzyskać ten sam wynik bez ręcznej minimalizacji. Przy tak małym układzie obie metody prowadzą do podobnej realizacji w tablicach LUT, natomiast blok IP jest wygodniejszy przy większej liczbie bitów lub innej stałej.